TaskFlow – Project Management Tool Documentation

Oct 9, 2026 · @ANBU

1. Project Introduction

TaskFlow is a project management website that runs completely inside the web browser. A user creates an account, logs in, and keeps a list of their projects. For each project they can see its dates and status, and update it at any time.

A **project management tool** is an app that helps people answer three simple questions: *What am I working on? What is finished? What is late?* Big companies use tools like Jira or Trello for this. TaskFlow is a small, simple version of the same idea, built by one developer as a learning and portfolio project.

Why it is useful:

- All projects are in one place instead of scattered in notes or chats.
- The dashboard shows totals at a glance: total, in progress, completed and pending.
- Each project has a deadline, and late projects show a red **Overdue** label.
- Search and filter help you find a project quickly.
- It needs no installation, no server and no internet connection for the data. Everything is saved in the browser.

Pages in the website: Login, Sign Up, Dashboard (with a Projects section), Profile, Settings and About.

2. Project Objectives

The main goal is to build a real, working web application using only the three core web languages, and to understand every line of it.

1. **Practise the basics:** use HTML for structure, CSS for design and JavaScript for behaviour, with no frameworks.
2. **Build CRUD features:** Create, Read, Update and Delete projects. CRUD is the base of almost every real app.
3. **Handle users:** sign up, log in, log out, edit a profile and change a password.
4. **Save data without a server:** use the browser's Local Storage so data stays after a page refresh.
5. **Validate forms:** check user input and show clear error messages.

6. **Make it responsive:** the site works on desktop, tablet and mobile.
7. **Make it look professional:** a modern dashboard design with cards, icons, shadows, hover effects and a dark/light theme.
8. **Create a portfolio piece:** a project a fresher can show and explain confidently in an interview.

3. Technologies Used

TaskFlow uses only four things: HTML, CSS, JavaScript and the browser's Local Storage. There is no backend, no database server and no framework.

A simple way to remember them: **HTML is the skeleton, CSS is the clothes, JavaScript is the brain, and Local Storage is the memory.**

Technology	What it is	Why TaskFlow uses it	Example in the project
HTML	The language that defines what is on a page: headings, forms, buttons, tables	To build the structure of all 6 pages	The login form in `index.html`, the sidebar in every app page
CSS	The language that controls how things look: colours, spacing, layout, animation	To make the site look like a modern SaaS dashboard and work on all screen sizes	Cards with shadows, the purple gradient banner, dark mode, mobile sidebar
JavaScript	The programming language that makes a page react to the user	To validate forms, log users in, add/edit/delete projects, search, filter and switch themes	`saveProject()`, `renderProjects()`, `applyTheme()` in `script.js`

Technology	What it is	Why TaskFlow uses it	Example in the project
Local Storage	A small storage space inside the browser that keeps text even after refresh or closing the browser	To save users, projects, login status and settings without a server	`localStorage.setItem("pm_projects", ...)`

Two small outside resources are loaded from the internet. They only add looks, not features:

- **Google Fonts (Inter):** the clean font used everywhere.
- **Font Awesome:** the icon library (folder, eye, pen, trash, moon and sun icons).

If the internet is off, the site still works, but icons do not show and a normal font is used.

4. Project Development Process

The project was built in small steps. Each step added one working piece, and each step was tested in the browser before moving on.

1. **Plan the pages and data.** Decide the pages (Login, Sign Up, Dashboard, Profile, Settings, About) and what data to store: users, projects, the logged-in user and settings.
2. **Create the folder and files.** One folder with 6 HTML files, one `style.css` and one `script.js`. Every page links the same CSS and JS file.
3. **Write the shared CSS first.** Colour variables (`--primary` , `--bg` , `--card` and so on), buttons, form fields and error styles. Because all pages share them, the design stays consistent.
4. **Build the Sign Up page.** HTML form with name, email, password and confirm password. JavaScript checks every field and saves the new user to Local Storage.
5. **Build the Login page.** Username/email and password fields, the show/hide password eye button, error messages, and the default admin account (`admin@123` / `123`). On success, the user's email is saved as `pm_currentUser` .
6. **Build the dashboard layout.** A fixed sidebar on the left and a top bar with the theme button, user avatar and Logout button.

7. **Add login protection.** Every app page has `<body data-page="...">`. If no one is logged in, the script sends the visitor back to `index.html`.
8. **Add the dashboard overview.** Four stat cards (Total, In Progress, Completed, Pending) and a Recent Projects table.
9. **Add project management (CRUD).** An Add/Edit popup form, project cards, View, Edit and Delete buttons, a status dropdown on each card, a delete confirmation, and the empty-state message.
10. **Add search and filter.** A search box and a status dropdown that redraw the cards as you type or choose.
11. **Add the Profile page.** Show user details, an Edit Profile popup and a Change Password popup.
12. **Add the Settings page.** Account info, a password form, a dark mode switch, the number of recent projects, a "delete all my projects" button and Logout.
13. **Add dark/light mode.** A second set of colour variables under `[data-theme="dark"]`, switched by JavaScript and saved in Local Storage.
14. **Add the About page.** A static page that explains the app's purpose and features.
15. **Make it responsive.** CSS media queries change the layout at 1024px, 860px and 560px widths. On small screens the sidebar slides in from a menu button.
16. **Test everything.** Try wrong inputs, empty forms, refreshing the page, logging out, and using a phone-sized screen.

5. Project Features

TaskFlow has nine main features. Each one is described below with what the user sees and what happens in the code.

Login

- Fields: **Username or Email** and **Password**.
- A demo box shows the default admin login: `admin@123` / `123`.
- Validation: both fields are required, and the username must contain `@`.
- Wrong details show the red message "Invalid username or password. Please try again."
- The eye button shows or hides the password.
- On success, the user's email is saved as `pm_currentUser` and the dashboard opens.
- If you are already logged in, the login page sends you straight to the dashboard.

Sign Up

- Fields: Full Name, Email, Password, Confirm Password.

- Rules: name at least 3 characters, a valid email, password at least 6 characters, both passwords must match, and the email must not already be registered.
- On success, the user is saved in `pm_users`, a green success message appears, and after 1.5 seconds the login page opens.

Dashboard

- A welcome banner: "Hello, *first name*" with an **Add Project** button.
- Four stat cards: **Total Projects, In Progress, Completed, Pending Tasks**.
- A **Recent Projects** table with the newest projects first. How many rows it shows (3, 5 or 10) comes from Settings.
- A "No projects yet" message when the user has no projects.

Project Management

- **Add:** a popup form with name, description, start date, deadline and status.
- **View:** a popup showing all details of one project.
- **Edit:** the same popup, filled with the project's current data.
- **Delete:** asks "Are you sure you want to delete ...?" before removing it.
- **Change status:** a dropdown on each card changes Pending / In Progress / Completed instantly.
- **Search:** matches the project name or description as you type.
- **Filter:** shows All, Pending, In Progress or Completed projects.
- **Overdue label:** a red "Overdue" tag appears when the deadline has passed and the project is not completed.
- **Empty messages:** "No projects available" when there are none, "No matching projects" when a search finds nothing.
- **Validation:** name, description and both dates are required, and the deadline cannot be before the start date.
- Each user only sees their own projects.

User Profile

- A profile card with a big avatar (first letter of the name), name, job title and email.
- Personal Information: full name, email, phone, job title, location, member since, and About Me.
- My Activity: the user's project counts.
- **Edit Profile** opens a popup to change name, email, phone, job title, location and bio. Phone is optional but must be 7 to 15 digits/spaces/+/-.

- If the email is changed, the user's projects move to the new email so nothing is lost.

Change Password

- Available on both the Profile page (as a popup) and the Settings page.
- Requires the **current password**, a **new password** (at least 6 characters and different from the current one), and **confirm new password** (must match).

Settings

- **Account Settings:** shows name, email and member-since date, with an Edit Profile button that opens the Profile page's edit popup (`profile.html?edit=1`).
- **Password Settings:** the change password form.
- **Appearance:** a dark mode switch and a "Recent Projects" count (3, 5 or 10).
- **Session & Data:** "Delete All My Projects" (with confirmation) and a Logout button.

Dark / Light Mode

- A moon/sun button in the top bar of every page, plus the switch on the Settings page.
- The choice is saved in `pm_settings` , so it stays the same on every page and after refresh.

About Page

- A banner with the app's purpose, an "Our Purpose" section, six feature cards, a "How It Works" list with 3 steps, and a "Built With" list of technologies.
- It is a static page: it only needs the shared sidebar, top bar and login protection.

6. Project Folder Structure

The whole project is one folder with 8 files and no sub-folders. All pages share one stylesheet and one script.

```
Project planning app/
├── index.html       Login page (first page that opens)
├── signup.html      Sign-up page
├── dashboard.html   Dashboard overview + Projects section + popups
├── profile.html     User profile + Edit Profile and Change Password popups
├── settings.html    Account, password, appearance and session settings
├── about.html       Information about the app
├── style.css        All the design for every page
└── script.js        All the JavaScript logic for every page
```

File	Lines (approx.)	Purpose	Main parts inside
`index.html`	93	Login page. The name `index.html` means browsers and servers open it first.	Brand panel, demo login hint, `loginForm`, eye button, link to Sign Up
`signup.html`	110	Create a new account	`signupForm` with 4 fields and error areas
`dashboard.html`	272	Main app screen	Sidebar, top bar, `dashboardSection` (stats + recent table), `projectsSection` (search, filter, cards), `projectModal` (add/edit), `viewModal`, toast
`profile.html`	269	Show and edit user details	Profile card, Personal Information, My Activity, `profileModal`, `passwordModal`
`settings.html`	222	User preferences	Account Settings, Password Settings, Appearance (switch + select), Session & Data
`about.html`	162	Explain the app	Hero banner, purpose, 6 feature cards, How It Works, Built With
`style.css`	1544	Design of every page, in 15 numbered sections	Colour variables, dark theme, buttons, forms, login pages, dashboard layout, cards, popups, profile, settings, about, responsive rules
`script.js`	1048	All behaviour, in 15 numbered sections	Local Storage helpers, validation helpers, admin account, theme, login, sign up, layout, dashboard CRUD, profile, password, settings, page start-up

How a page and the script know each other: the login and sign-up pages are recognised by their form ids (`loginForm` , `signupForm`). The four app pages have a `data-page` attribute on the `<body>` tag, for example `<body class="dash-body" data-page="profile">` . The last section of `script.js` reads this value and runs the right code for that page.

7. Code Explanation

`script.js` is split into 15 numbered sections. The table shows what each one does; the parts after it explain the most important ideas with real code from the project.

#	Section	What it does
1	Local Storage helpers	Read and save users, projects and settings; find the logged-in user
2	Small helper functions	Check email, show/clear errors, alerts, toast messages, open/close popups, safe text, format dates
3	Default admin account	Adds `admin@123` / `123` once
4	Dark / light theme	Applies and saves the theme
5	Show / hide password	Makes every eye button work
6	Sign-up page	Validates and saves a new user
7	Login page	Validates, checks details and saves login status
8	Shared layout	Top bar user info, mobile menu, logout, closing popups
9	Dashboard setup	Sidebar section switching, search/filter listeners
10	Dashboard drawing	Stat cards, recent table, project cards
11	Project actions	Add, edit, view, delete, change status
12	Profile page	Show and edit profile
13	Change password	One function used by Profile and Settings
14	Settings page	Theme switch, recent count, delete all projects
15	Page start-up	Login protection and running the right page code

Functions

A **function** is a named block of code you can run again and again. TaskFlow uses many small functions, each with one job.

```
// Turn "In Progress" into the CSS class "in-progress"
function statusClass(status) {
  return status.toLowerCase().replace(" ", "-");
}
```

`statusClass("In Progress")` returns `"in-progress"`. The CSS then colours the badge blue with `.badge.in-progress`.

Variables

A **variable** is a named box that holds a value. The project uses `const` for values that never change and `let` for values that do.

```
const themeButton = document.getElementById("themeToggle"); // never changes
let currentUser = null; // filled in later, after we know who is logged in
let isValid = true;     // becomes false if any field has an error
```

Event listeners

An **event** is something the user does: a click, typing, submitting a form. An **event listener** waits for that event and then runs a function.

```
document.getElementById("searchInput").addEventListener("input",
renderProjects);
document.getElementById("statusFilter").addEventListener("change",
renderProjects);
```

Every time the user types in the search box (`input` event) or picks a status (`change` event), `renderProjects` runs and redraws the cards.

The project cards use a simpler style called an **inline handler**: `onclick="deleteProject(123)"` written straight into the button's HTML.

DOM manipulation

The **DOM** (Document Object Model) is the browser's live copy of the HTML page that JavaScript can read and change. Changing it updates the screen instantly.

```
document.getElementById("totalCount").textContent = counts.total;   // change
text
emptyBox.classList.add("show");                                      // add a
CSS class
grid.innerHTML += '<div class="project-card">...</div>';             // add
new HTML
```

- `getElementById` finds one element by its `id`.
- `querySelectorAll` finds all elements that match a CSS selector, e.g. every `.logout-btn`.
- `textContent` sets plain text. `innerHTML` sets HTML.
- `classList.add / remove / toggle` switch CSS classes on and off. Popups open by adding the class `show`.

Form validation

Every form follows the same pattern: stop the page reloading, clear old errors, check each field, stop if anything is wrong, otherwise save.

```
loginForm.addEventListener("submit", function (event) {
  event.preventDefault();     // stop the page from reloading
  clearErrors(loginForm);     // remove old red messages

  const username =
document.getElementById("loginEmail").value.trim().toLowerCase();
  const password = document.getElementById("loginPassword").value;
  let isValid = true;

  if (username === "") {
    showError("loginEmail", "Please enter your username or email.");
    isValid = false;
  }
  ...
  if (!isValid) return;       // stop here if any error was found
  ...
});
```

`showError("loginEmail", "...")` puts a red border on the input and writes the message into the `<small>` element whose id is `loginEmailError`. That naming rule (input id + `Error`) is used on every form.

Local Storage operations

Local Storage stores only **text**. Arrays and objects are turned into text with `JSON.stringify` before saving, and back into arrays with `JSON.parse` when reading.

```
function getAllProjects() {
  return JSON.parse(localStorage.getItem("pm_projects")) || [];
}

function saveAllProjects(projects) {
  localStorage.setItem("pm_projects", JSON.stringify(projects));
}
```

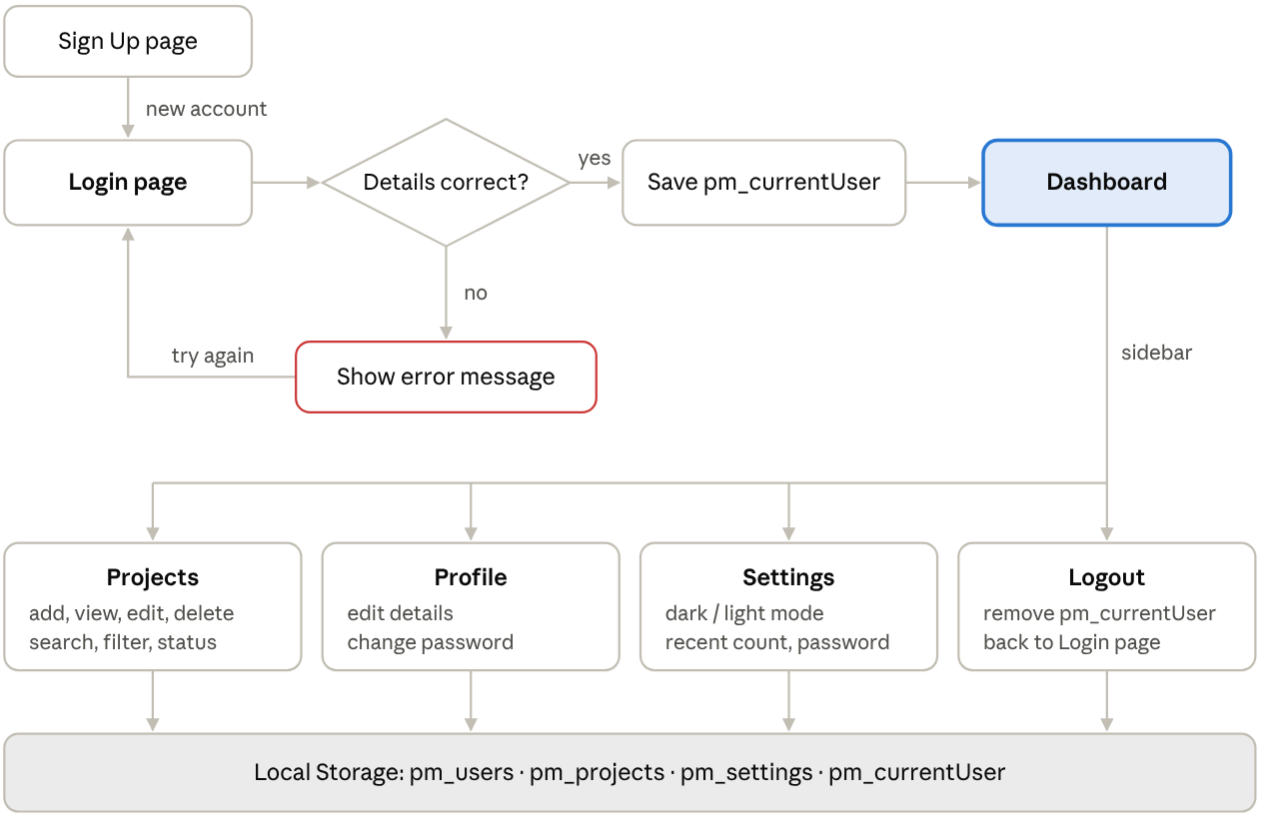
The `|| []` part means: if nothing is saved yet, use an empty list instead of `null`.

Array methods used

Method	What it does	Example in TaskFlow
`push`	Adds an item to the end	Add a new user or project
`find`	Returns the first matching item	Find the user with this email and password
`filter`	Returns all matching items	`getMyProjects()`, search and filter, delete
`some`	True if at least one item matches	Is this email already registered?
`forEach`	Runs code for every item	Build a card for each project
`sort`	Puts items in order	Newest projects first (`b.id - a.id`)
`slice`	Takes part of a list	Only the first 3, 5 or 10 recent projects
`findIndex`	Returns the position of a match	Replace the edited user in `saveCurrentUser`

8. Application Workflow

A user always passes through the login page first; every other page checks for a logged-in user before showing anything.



TaskFlow user flow · login check, 4 app areas, Local Storage

Wrong details loop back to the Login page; once logged in, the sidebar leads to every area, and each change is saved to Local Storage.

1. **Open the site.** `index.html` (Login) opens. The script adds the admin account the first time.
2. **Sign up (new users).** Fill in the form on `signup.html`. If all checks pass, the user is saved and sent back to Login after 1.5 seconds.
3. **Log in.** Enter username/email and password. The script looks for a user in `pm_users` with the same email *and* password.
   - Not found: a red error message appears and the user stays on Login.
   - Found: the email is saved as `pm_currentUser` and `dashboard.html` opens.
4. **Login protection.** Every app page runs the start-up code at the bottom of `script.js`. If `pm_currentUser` is empty, the visitor is sent back to `index.html`.
5. **Dashboard.** The top bar shows the user's name and avatar. Stat cards and the Recent Projects table are filled from the user's projects.
6. **Manage projects.** Click **Projects** in the sidebar (or "View all"). Add a project with the popup form, change status with the card dropdown, view, edit or delete. Search and filter to find projects. Every change is saved to `pm_projects` at once, and the stats

update.

7. **Update the profile.** Open **Profile**, click **Edit Profile**, change details and save. The top bar and profile card update straight away.
8. **Change the password.** On Profile (Change Password button) or Settings (Password Settings). Enter the current password, a new one and the confirmation.
9. **Change settings.** Turn dark mode on or off, or choose 3, 5 or 10 recent projects. Settings are saved in `pm_settings`.
10. **Log out.** Click Logout in the sidebar, top bar or Settings. After "Are you sure you want to log out?", `pm_currentUser` is removed and the Login page opens. The user's projects stay saved for next time.

9. Local Storage Explanation

TaskFlow saves all its data under 5 keys in the browser's Local Storage. Think of Local Storage as a small notebook inside the browser: each page has a **name (key)** and some **text (value)**.

Key	What it holds	Example value (shortened)	Written when
`pm_users`	List of all user accounts	`[{"name":"Admin User","email":"admin@123","password":"123","role":"Administrator",...}]`	Sign up, edit profile, change password
`pm_currentUser`	Email/username of the logged-in user (the login status)	`admin@123`	Login (set), Logout (removed), email change
`pm_projects`	List of all projects of all users	`[{"id":1791546560693,"owner":"admin@123","name":"Website Redesign","status":"In Progress",...}]`	Add, edit, delete, change status, delete all
`pm_settings`	Preferences	`{"theme":"dark","recentCount":5}`	Theme button, dark mode switch, recent count
`pm_adminCreated`	A flag that the admin was already added	`yes`	First page load

The 4 Local Storage commands used

```
localStorage.setItem("pm_currentUser", user.email);   // save
localStorage.getItem("pm_currentUser");               // read
localStorage.removeItem("pm_currentUser");            // delete one key
(logout)
JSON.stringify(list) / JSON.parse(text)               // convert list <->
text
```

What one user object looks like

```
{
  name: "Admin User",
  email: "admin@123",
  password: "123",
  phone: "",
  role: "Administrator",
  location: "",
  bio: "Default administrator account.",
  joined: "2026-10-09"
}
```

What one project object looks like

```
{
  id: 1791546560693,         // Date.now() - a unique number from the current
time
  owner: "admin@123",        // which user this project belongs to
  name: "Website Redesign",
  description: "Refresh the marketing site",
  startDate: "2026-09-01",
  deadline: "2026-11-30",
  status: "In Progress"      // Pending, In Progress or Completed
}
```

How data is kept separate for each user

All projects live in one list, but each has an `owner`. `getMyProjects()` keeps only the ones where `owner` equals the logged-in user's email:

```
function getMyProjects() {
  return getAllProjects().filter(function (project) {
    return project.owner === currentUser.email;
  });
}
```

How to see the data yourself

Press **F12** in the browser, open the **Application** tab, then **Local Storage** and click the site address. You will see all the `pm_` keys and their values. Deleting them resets the app; the admin account is added again on the next page load.

10. How to Run the Project

The project needs no installation or build step; any modern browser can open it. VS Code with the Live Server extension is the easiest way.

1. **Install VS Code** from code.visualstudio.com if you do not have it.
2. **Open the folder:** File → Open Folder → choose `Project planning app`.
3. **Install Live Server:** press Ctrl+Shift+X, search for "Live Server" (by Ritwick Dey) and click Install.
4. **Start the site:** right-click `index.html` in the file list and choose **Open with Live Server**. The browser opens the Login page, usually at `http://127.0.0.1:5500/index.html`.
5. **Log in:** use `admin@123` / `123`, or click **Sign Up** to create your own account.
6. **Edit and see changes:** when you save a file in VS Code, Live Server reloads the page automatically.

Without VS Code: double-click `index.html` and it opens in your default browser. Everything works the same way.

Good to know:

- The icons and font need an internet connection. The app's data does not.
- Data is saved per browser. Chrome and Edge, or Live Server and double-click (different addresses), each keep their own separate data.
- To reset the app, delete the `pm_` keys in DevTools (F12 → Application → Local Storage).

11. Project Advantages and Limitations

A frontend-only app is simple, free and fast to build, but it cannot share data between devices or keep data truly secure.

Advantages	Limitations
No server, database or hosting cost	Data lives in one browser on one device; it does not sync to a phone or another computer
Opens instantly, works offline (except icons and font)	Clearing browser data deletes all users and projects
Easy to understand: only HTML, CSS and JavaScript	Passwords are stored as plain text, so anyone with access to the browser can read them in DevTools

Advantages	Limitations
Easy to host for free (GitHub Pages, Netlify) as static files	"Login" is only a front-end check; a user could edit Local Storage and pretend to be someone else
Data stays after refresh thanks to Local Storage	Local Storage holds only about 5 MB per site and only text
Fully responsive with a dark/light theme	Users cannot share projects or work as a team
Shows the core skills every web developer needs: DOM, events, validation, CRUD, storage	No automatic tests, and no email for password recovery

How it could be improved later (ideas, not in the current code): a backend with Node.js or another server language, a real database, hashed passwords, team sharing, task lists inside each project, and deadline reminders.

12. Interview Preparation

These are the questions interviewers most often ask about a project like this, with short answers in simple words. Practise saying them out loud.

About the project

Q1. Tell me about your project. TaskFlow is a project management website built with HTML, CSS and JavaScript. Users sign up, log in and manage their projects: add, view, edit, delete, change status, search and filter. It has a dashboard with statistics, a profile page, a settings page with dark mode, and all data is saved in the browser's Local Storage.

Q2. Why did you build this project? To practise core web development without frameworks. It covers the most common real-world features: login, forms, validation, CRUD and saving data.

Q3. Why didn't you use React or a backend? I wanted to understand the basics first: how the DOM, events and storage work. Frameworks are built on these ideas, so learning them first makes React and Node.js easier later.

Q4. What was the most challenging part? Keeping the data correct across pages. For example, when a user changes their email in the profile, their projects are linked to the old email. I solved it by updating the `owner` of every project from the old email to the new one before saving.

JavaScript and DOM

Q5. What is the DOM? The DOM is the browser's live version of the HTML page. JavaScript can read and change it, for example `document.getElementById("totalCount").textContent = 4` changes the number on screen.

Q6. What is an event listener? Code that waits for a user action and then runs a function. Example: `addEventListener("submit", ...)` on the login form runs the login check when the form is sent.

Q7. Why do you use `event.preventDefault()`? A form normally reloads the page when submitted. `preventDefault()` stops that, so my JavaScript can check the inputs and show errors instead.

Q8. What is the difference between `let` and `const`? `const` cannot be given a new value later; `let` can. I use `const` for things like page elements and `let` for values that change, like `isValid` or `currentUser`.

Q9. What is the difference between `textContent` and `innerHTML`? `textContent` sets plain text. `innerHTML` reads the text as HTML. I use `innerHTML` to build project cards, but first I pass user text through `escapeHTML()` so typed tags like `<script>` show as text and cannot run.

Q10. How do search and filter work? On every keypress or dropdown change, `renderProjects()` runs. It uses `filter()` to keep projects whose name or description contains the search text and whose status matches the filter, then rebuilds the cards.

Local Storage and login

Q11. What is Local Storage? A small storage space in the browser that saves text as key–value pairs. The data stays after refreshing or closing the browser.

Q12. How do you store an array in Local Storage? Local Storage only stores strings, so I convert the array with `JSON.stringify()` before saving and convert it back with `JSON.parse()` when reading.

Q13. What is the difference between Local Storage and Session Storage? Local Storage keeps data until it is deleted. Session Storage is cleared when the browser tab closes. I used Local Storage so users and projects are not lost.

Q14. How does your login work? The script searches the users list for a user with the same email and password. If found, it saves that email as `pm_currentUser`. Each app page checks this key when it loads; if it is empty, the user is sent back to the login page.

Q15. Is your login secure? No, and I know why. Everything runs in the browser and passwords are plain text in Local Storage. In a real app I would use a backend, hash passwords and use secure sessions or tokens. This project is for learning the front-end flow.

Q16. How does each user see only their own projects? Every project has an `owner` field with the user's email. `getMyProjects()` filters the list to projects where `owner` matches the logged-in user.

Design

Q17. How did you make it responsive? With CSS Grid, Flexbox and media queries at 1024px, 860px and 560px. On small screens the grids become one column and the sidebar hides off-screen; the menu button slides it in.

Q18. How does dark mode work? Colours are CSS variables like `--bg` and `--text`. A second set is defined under `[data-theme="dark"]`. JavaScript sets `data-theme` on the `<html>` tag and saves the choice in `pm_settings`, so it stays on every page.

Q19. How do you validate forms? Each submit handler checks every field (empty, length, email format, matching passwords, dates in order). If something is wrong, `showError()` adds a red border and a message under that field, and the save is stopped.

Q20. What would you add next? A real backend and database, hashed passwords, team sharing, and tasks inside each project.

13. Project Presentation

Use this 5-minute script with the website open in the browser. Each step says what to **click** and what to **say**. You do not need to show code unless they ask.

Before the interview: open the site, log in once as admin, add 3 or 4 sample projects with different statuses (one with a past deadline so "Overdue" shows), then log out.

1. **Introduce (30 seconds).**
   - *Say:* "My project is TaskFlow, a project management website. I built it with only HTML, CSS and vanilla JavaScript, and it stores all data in the browser's Local Storage. Users can manage their projects, track status and deadlines, and update their profile."
2. **Login page (30 seconds).**
   - *Click:* Log In with empty fields to show the errors. Click the eye icon. Then log in with `admin@123` / `123`.

- *Say:* "The form is validated with JavaScript. There is a default admin account, and new users can sign up. After login, I save the user in Local Storage, and every page checks it, so you cannot open the dashboard without logging in."

3. **Dashboard (45 seconds).**
   - *Point at* the stat cards and Recent Projects table.
   - *Say:* "The dashboard counts the user's projects by status and shows the most recent ones. These numbers are calculated by JavaScript every time the data changes."
4. **Projects – CRUD (90 seconds).**
   - *Click:* Projects → Add Project. Try saving with a deadline before the start date to show the error. Fix it and save.
   - *Click:* change a status with the dropdown, then View, Edit and Delete (show the confirmation).
   - *Type* in the search box and use the status filter.
   - *Say:* "This is full CRUD: create, read, update and delete. Every change is saved to Local Storage immediately, so if I refresh…" *(press F5)* "…the data is still here. Overdue projects get a red label."
5. **Profile and password (45 seconds).**
   - *Click:* Profile → Edit Profile, change the job title, save. Open Change Password and show the "current password is incorrect" error.
   - *Say:* "Profile details are saved per user. Changing the password requires the current password, and the new one is validated."
6. **Settings and dark mode (30 seconds).**
   - *Click:* Settings → turn on Dark Mode. Go to another page to show it stays.
   - *Say:* "I used CSS variables for colours, so dark mode is just a second set of variables. The choice is saved in Local Storage."
7. **Responsive design (20 seconds).**
   - *Do:* press F12 → the phone icon (device toolbar) to show the mobile view and open the sidebar menu.
   - *Say:* "It is responsive using CSS Grid, Flexbox and media queries."
8. **Close (20 seconds).**
   - *Click:* Logout.
   - *Say:* "This project taught me DOM manipulation, events, form validation and Local Storage. Its limitation is that data stays in one browser and passwords are not secure, so my next step is to add a backend and database."

If you get stuck on a code question: say what the feature does first, then name the function from this document. Example: "Search is handled by the `renderProjects` function. It filters the project list by the search text and status, and redraws the cards." It is fine to say "Let me open the file and show you" and find the function by name with Ctrl+F.

14. Code Explanation for Beginners

This section walks through the most important pieces of code line by line. If you can explain these seven pieces, you can explain the whole project.

Glossary: technical words in simple English

Word	Simple meaning	Where you see it in TaskFlow
Element	One piece of a page, like a button or input	`<button id="themeToggle">`
Attribute	Extra information on an element	`id`, `class`, `data-page`, `data-target`
id	A unique name for one element	`id="loginForm"`
class	A group name; many elements can share it	`class="logout-btn"` on 3 buttons
Function	Named block of code you can run many times	`showToast("Project deleted")`
Parameter	A value you give to a function	`message` in `showToast(message)`
Return	The answer a function gives back	`return status.toLowerCase()...`
Array	A list of items in `[ ]`	The list of projects
Object	A group of named values in `{ }`	One project: `{ name, status, ... }`
String	Text in quotes	`"In Progress"`
Boolean	`true` or `false`	`isValid`
Condition	A check with `if` / `else`	`if (password === "")`
`===`	"Is exactly equal to"	`u.email === email`
`!`	"Not"	`if (!isValid) return;`

Word	Simple meaning	Where you see it in TaskFlow
`&&` / `||`	"And" / "Or"	`u.email === username && u.password === password`
Callback	A function given to another function to run later	`function (event) { ... }` inside `addEventListener`
JSON	A text format for arrays and objects	`JSON.stringify(projects)`
CSS variable	A named value reused in CSS	`--primary: #4f46e5;`
Media query	CSS that applies only at certain screen widths	`@media (max-width: 860px)`
Modal / popup	A box that appears over the page	Add Project form
Toast	A small message that disappears by itself	"Project added successfully"

Example 1: Finding the logged-in user

```
function getCurrentUser() {
  const email = localStorage.getItem("pm_currentUser");
  if (!email) return null;
  const users = getUsers();
  return users.find(function (user) { return user.email === email; }) ||
null;
}
```

1. `function getCurrentUser()` creates a function with that name.
2. Read the saved login email from Local Storage into `email`.
3. If there is no email (nobody logged in), stop and give back `null` (meaning "nothing").
4. Get the full list of users.
5. `find` looks through the list and returns the first user whose email matches. If none matches, return `null`.

Real-life example: like a receptionist who checks the visitor badge (email) and then finds that person's file in the cabinet (users list).

Example 2: Adding the admin account only once

```
if (!localStorage.getItem("pm_adminCreated")) {
  const users = getUsers();
  users.push({ name: "Admin User", email: "admin@123", password: "123", ...
});
  saveUsers(users);
  localStorage.setItem("pm_adminCreated", "yes");
}
```

1. If the flag `pm_adminCreated` does **not** exist yet…
2. …get the users list,
3. add the admin user to the end with `push`,
4. save the list,
5. and save the flag `"yes"` so this never runs again. Because of the flag, if the admin changes the password, the old one is not added back.

Example 3: Show / hide password

```
document.querySelectorAll(".toggle-password").forEach(function (button) {
  button.addEventListener("click", function () {
    const input = document.getElementById(button.dataset.target);
    const icon = button.querySelector("i");

    if (input.type === "password") {
      input.type = "text";
      icon.className = "fa-regular fa-eye-slash";
    } else {
      input.type = "password";
      icon.className = "fa-regular fa-eye";
    }
  });
});
```

1. Find every eye button (class `toggle-password`) and repeat the next steps for each one.
2. When a button is clicked…
3. …read its `data-target` attribute (for example `loginPassword`) and find that input.
4. Find the icon inside the button.
5. If the input is hidden (`type="password"`), change it to `type="text"` so the letters show, and switch to the crossed-eye icon.
6. Otherwise, hide it again and show the normal eye icon.

One piece of code works for all 9 eye buttons in the project (1 on Login, 2 on Sign Up, 3 on Profile, 3 on Settings) because each button says which input it controls.

Example 4: Saving a new or edited project

```
function saveProject(event) {
  event.preventDefault();
  ...read the 6 fields and validate them...
  const projects = getAllProjects();

  if (id === "") {
    projects.push({
      id: Date.now(),
      owner: currentUser.email,
      name: name, description: description,
      startDate: startDate, deadline: deadline, status: status
    });
    showToast("Project added successfully");
  } else {
    const project = projects.find(function (p) { return p.id === Number(id);
});
    project.name = name;
    ...
    showToast("Project updated successfully");
  }

  saveAllProjects(projects);
  closeModal("projectModal");
  refreshAll();
}
```

- The same popup is used for **Add** and **Edit**. A hidden input `projectId` tells them apart: empty means Add, a number means Edit.
- `Date.now()` gives the number of milliseconds since 1970. It is different every time, so it works as a unique id.
- `Number(id)` turns the text from the hidden input into a number, because ids are saved as numbers.
- Changing `project.name` changes the object inside the `projects` list, so saving the list saves the change.
- After saving: close the popup and redraw the stats, table and cards.

Example 5: Deleting with confirmation

```
function deleteProject(id) {
  const project = findProject(id);
  if (!project) return;
  if (!confirm('Are you sure you want to delete "' + project.name + '"?'))
return;

  const projects = getAllProjects().filter(function (p) { return p.id !== id;
});
  saveAllProjects(projects);
  refreshAll();
  showToast("Project deleted");
}
```

1. Find the project; stop if it does not exist.
2. `confirm()` shows the browser's OK/Cancel box. If the user clicks Cancel it returns `false`, and `!false` is `true`, so the function stops.
3. `filter` keeps every project **except** the one with this id. That is how delete works: make a new list without it.
4. Save, redraw, and show a toast.

Example 6: Dark mode

CSS (in `style.css`):

```
:root {
  --bg: #f4f6fb;
  --text: #1e293b;
}
[data-theme="dark"] {
  --bg: #0f172a;
  --text: #e2e8f0;
}
body {
  background: var(--bg);
  color: var(--text);
}
```

JavaScript (in `script.js`):

```
function applyTheme() {
  const theme = getSettings().theme;
  document.documentElement.setAttribute("data-theme", theme);
  ...
}
```

- `:root` holds the light colours. `[data-theme="dark"]` replaces them when the `<html>` tag has `data-theme="dark"`.
- Every part of the design uses `var(--bg)`, `var(--text)` and so on, so switching the variables recolours the whole site at once.
- `document.documentElement` means the `<html>` tag. `applyTheme()` runs on every page load, so the saved theme is applied everywhere.

Example 7: Login protection and page start-up

```
const appPage = document.body.dataset.page;

if (appPage) {
  currentUser = getCurrentUser();

  if (!currentUser) {
    window.location.href = "index.html";
  } else {
    setupLayout();
    setupPasswordForm();

    if (appPage === "dashboard") startDashboard();
    if (appPage === "profile") startProfile();
    if (appPage === "settings") startSettings();
  }
}
```

1. Read `data-page` from the `<body>` tag (`"dashboard"`, `"profile"`, `"settings"` or `"about"`). Login and sign-up pages do not have it, so this block does nothing there.
2. Find the logged-in user.
3. If nobody is logged in, `window.location.href` changes the address and opens the login page.
4. Otherwise, set up the shared parts (top bar, menu, logout, popups) and the password form if the page has one.
5. Run only the start function for the current page. The About page needs nothing extra.

Tips for reading the code yourself

- Start at the bottom of `script.js` (section 15) and follow which function each page calls.
- In VS Code, hold **Ctrl** and click a function name to jump to where it is written.
- Press **F12** → **Console** and type `getUsers()` or `getAllProjects()` while on the site to see the data.
- Change one small thing (a colour in `:root`, or a toast message), save, and watch what changes.